

Reviewer Pack — Minimal Rank of p-adic L-Functions Bounds Disjointness Commu...

Entry 9c789c69dba6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 19:16:31 UTC

Minimal Rank of p-adic L-Functions Bounds Disjointness Communication Complexity

Entry ID: 9c789c69dba6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 19:16:31 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (L-Functions) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

[‘For any fixed prime p , there exists a constant $C_p > 0$ such that for all $n \leq 40$ and any communication protocol P for the disjointness function Disj_n with error probability less than $2^{(-n/2)}$, the minimal rank of the associated p-adic L-function $L(P)$ is at least $C_p * \log(n)$.’, ‘Equivalently, the lower bound on the randomized communication complexity of Disj_n can be characterized by an upper bound on the rank of a certain p-adic L-function.’, ‘Furthermore, for any counterexample to this conjecture, there exists a communication protocol P' for Disj_n with minimal rank less than $C_p * \log(n)$, which would allow for a polynomial-time algorithm to solve Disj_n with high probability.”]

Rationale (proposer’s reasoning):

[‘p-adic L-functions are known to encode deep arithmetic information about number fields, and their ranks are related to the algebraic properties of these functions. By connecting the rank of p-adic L-functions to communication complexity, this conjecture aims to reveal a previously unknown link between arithmetic geometry and computational complexity.’, ‘If true, it would provide a new tool for understanding the inherent difficulty of the disjointness problem, which is a fundamental problem in communication complexity.’, ‘It also suggests that p-adic L-function techniques could potentially be used to analyze other problems in complexity theory.’]

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fa239552ba14d2ee

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For Disj_n with $n \leq 40$ and error probability $< 2^{-(n/2)}$, if all 30 constructed p-adic L-functions $L(P)$ have a rank $\geq C_p * \log(n)$ for some constant $C_p > 0$, the conjecture is supported. Otherwise, it is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic L-functions AND communication complexity AND Disjointness - rank of p-adic L-functions IN relation TO Disjointness communication complexity - upper bound on rank of p-adic L-function IN terms of Disj_n communication complexity

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/2408.00810v3>] p-adic Equiangular Lines and p-adic van Lint-Seidel Relative Bound - [<http://arxiv.org/abs/hep-th/9410058v3>] p-Adic description of Higgs mechanism I: p-Adic square root and p-adic light cone - [<http://arxiv.org/abs/1204.3878v1>] Analytic constructions of p-adic L-functions and Eisenstein series - [<http://arxiv.org/abs/1901.01957v6>] (p -adic) L -functions and (p -adic) (multiple) zeta values - [<http://arxiv.org/abs/1601.00010v1>] On pairs of p-adic L-functions for weight two modular forms - [<http://arxiv.org/abs/2407.06983v4>] On the p -adic L -function and Iwasawa Main Conjecture for an Artin motive over a CM field

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k = len(A), len(B)
    n = len(B[0])
    C = [[0] * n for _ in range(m)]
    for i in range(m):
```

```

        for j in range(n):
            for l in range(k):
                C[i][j] += A[i][l] * B[l][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(augmented[j][i]) > abs(augmented[max_row][i]):
                max_row = j
        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]
        pivot = augmented[i][i]
        for j in range(n + 1):
            augmented[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = augmented[j][i]
                for k in range(n + 1):
                    augmented[j][k] -= factor * augmented[i][k]
    return [row[-1] for row in augmented]

def rank(matrix):
    A = matrix[:]
    m, n = len(A), len(A[0])
    r = 0
    for i in range(n):
        if r < m:
            pivot_row = r
            while pivot_row < m and A[pivot_row][i] == 0:
                pivot_row += 1
            if pivot_row < m:
                A[r], A[pivot_row] = A[pivot_row], A[r]
                for j in range(n):
                    A[r][j] /= A[r][i]
                for j in range(m):
                    if j != r:
                        factor = A[j][i]
                        for k in range(n):
                            A[j][k] -= factor * A[r][k]
            r += 1
    return r

def run_trial(seed: int) -> dict:
    random.seed(seed)
    p = 2 # Fixed prime p
    n_values = [5, 10, 15, 20, 30, 40]
    C_p = 1.0 # Placeholder constant for demonstration purposes

    results = []
    for n in n_values:
        instances_tested = 0
        conjecture_holds = True
        counterexample = ""

```

```

for _ in range(5): # Sample 5 instances per n
    A = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    b = [sum(A[i][j] for j in range(n)) % 2 for i in range(n)]

    rank_Ab = rank(gaussian_elimination(A, b))
    expected_rank = C_p * math.log(n)

    if rank_Ab < expected_rank:
        conjecture_holds = False
        counterexample = f"n={n}, rank={rank_Ab}, expected>={expected_rank}"
        break

    instances_tested += 1

results.append({
    "metric_name": "p-adic L-function rank",
    "metric_value": expected_rank,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
})

return {
    "seed": seed,
    **results[-1]
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    std_value = math.sqrt(sum((r['metric_value'] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results):
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9282c806.py", line 121, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9282c806.py", line 92, in run_trial
    rank_Ab = rank(gaussian_elimination(A, b))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9282c806.py", line 47, in gaussian_elim
    augmented[i][j] /= pivot
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test without errors to validate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12953
2	preregistration	ollama_remote	glm4:latest	0	0	5684
3	novelty	ollama_remote	glm4:latest	0	0	4756
4	novelty	ollama_remote	glm4:latest	0	0	9306
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36338
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9215
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7261
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13421
9	judge	ollama_remote	glm4:latest	0	0	12392

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 111324 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9c789c69dba6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9c789c69dba6.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9c789c69dba6.tar.gz` (if generated)